

End-to-End Use Case: Implementing CI/CD for Azure Data Factory Pipeline Using Azure DevOps

Contents

1	Problem Statement	2
2	Solution Overview	2
3	Task Assignments	2
4	Detailed Steps	3
4.1	Person 11: Create Azure Resources	3
4.2	Person 12: Set Up Azure DevOps Repo and Configure Git Integration in ADF .	4
4.3	Person 13: Create the ADF Pipeline and Datasets in a Feature Branch	5
4.4	Person 14: Set Up the CI Build Pipeline in Azure DevOps	6
4.5	Person 15: Set Up the CD Release Pipeline in Azure DevOps	7
4.6	Person 16: Test the Pipeline, Make a Change, Create Pull Request, Approve/Merge, and Validate	8
5	Collaboration Notes	9

1 Problem Statement

Develop a simple data ingestion pipeline in Azure Data Factory (ADF) that copies data from an Azure Blob Storage CSV file to an Azure SQL Database table. Implement CI/CD using Azure DevOps to manage ADF artifacts with Git version control, validate changes through a build pipeline, and deploy to production via a release pipeline. This demonstrates the software development lifecycle for data projects, including collaboration, version control, testing, and automated deployment.

The pipeline will:

- Read a sample CSV file from Blob Storage.
- Copy it to a SQL table.
- Use branches for development (feature branch) and production (main branch).
- Trigger CI on pull request to main.
- Trigger CD on merge to main.

Assumptions:

- Access to an Azure subscription.
- Access to an Azure DevOps organization and project.
- An Azure Resource Manager service connection named "AzureServiceConnection" exists in Azure DevOps, with Contributor permissions on the subscription.
- All participants collaborate in the same Azure DevOps project and Azure resource group.
- Use a single ADF instance for simplicity, with parameters to simulate dev/prod environments (e.g., different storage/SQL in parameters, but for this demo, use the same).
- Total time: Approximately 2 hours with parallel tasks.

2 Solution Overview

1. Create necessary Azure resources (Storage, SQL, ADF).
2. Set up Azure DevOps Git repo and integrate with ADF.
3. Develop the ADF pipeline in a feature branch.
4. Create a CI build pipeline to validate ADF ARM templates.
5. Create a CD release pipeline to deploy the ADF pipeline.
6. Test the end-to-end flow by making changes, committing, pulling requests, and deploying.

The tasks are divided among 6 participants (Person 11 to Person 16) for collaboration. Each person focuses on their task but coordinates (e.g., via shared screens or chats) for dependencies. Use Azure portal for resource creation, ADF Studio for pipeline development, and Azure DevOps for pipelines.

3 Task Assignments

- **Person 11:** Create Azure resources (Blob Storage, SQL Database, ADF).

- **Person 12:** Set up Azure DevOps repo and configure Git integration in ADF.
- **Person 13:** Create the ADF pipeline and datasets in a feature branch.
- **Person 14:** Set up the CI build pipeline in Azure DevOps.
- **Person 15:** Set up the CD release pipeline in Azure DevOps.
- **Person 16:** Test the pipeline, make a change, create a pull request, approve/merge, and validate deployment.

4 Detailed Steps

4.1 Person 11: Create Azure Resources

1. Log in to the Azure Portal (<https://portal.azure.com>).
2. Create a Resource Group:
 - Search for "Resource groups".
 - Click "Create".
 - Subscription: Select your subscription.
 - Resource group: "adf-demo-rg".
 - Region: East US (or any).
 - Click "Review + create" > "Create".
3. Create Azure Storage Account:
 - In the resource group "adf-demo-rg", click "Create a resource" > Search "Storage account" > Create.
 - Basics: Subscription and RG as above, Name: "adfdemostorage" (unique), Region: Same as RG, Performance: Standard, Redundancy: LRS.
 - Click "Review + create" > "Create".
 - After creation, go to the storage account > "Containers" > Create container named "inputdata".
 - Upload a sample CSV file to "inputdata":
 - Create a file locally named "sampledata.csv" with content:

Id	Name	Age
1	John	30
2	Jane	25
 - Upload it to the "inputdata" container.
4. Create Azure SQL Database:
 - In RG "adf-demo-rg", create "SQL server":
 - Search "SQL servers" > Create.
 - Server name: "adfdemosqlserver" (unique), Location: Same, Authentication: SQL login, Admin login: "sqladmin", Password: "P@ssw0rd123!" (note it down).

- Firewall: Add your IP.
- Click "Review + create" > "Create".
- Create database:
 - In the SQL server, "Create database".
 - Database name: "adfdemodb".
 - Compute: Basic (cheapest for demo).
 - Click "Create".
- Connect to the database using Azure Data Studio or SSMS:
 - Server: adfdemosqlserver.database.windows.net
 - Login: sqladmin / P@ssw0rd123!
 - Create table:

```
CREATE TABLE SampleData (
    Id INT,
    Name VARCHAR(50),
    Age INT
);
```

5. Create Azure Data Factory:

- In RG "adf-demo-rg", search "Data factories" > Create.
- Name: "adf-demo".
- Region: Same.
- Click "Review + create" > "Create".

6. Share the resource names, SQL credentials (securely), and RG with the team. Wait for Person 12 to proceed.

Time estimate: 20-30 minutes.

4.2 Person 12: Set Up Azure DevOps Repo and Configure Git Integration in ADF

1. Log in to Azure DevOps (<https://dev.azure.com>).
2. Assume an existing project named "ADF-Demo-Project". If not, create one:
 - Organization > New project > Name: "ADF-Demo-Project", Visibility: Private.
3. Create a Git Repo:
 - In the project, go to "Repos" > "New repository".
 - Repository name: "ADF-Repo".
 - Add a .gitignore: Select "AzureDataFactory".
 - Create.
4. Configure Git in ADF:
 - Go to Azure Portal > ADF "adf-demo" > Launch Studio (<https://adf.azure.com>).

- In ADF Studio, click "Manage" (gear icon) > "Source control" > "Git configuration" > "Configure".
 - Git type: Azure DevOps Git.
 - Azure Active Directory: Your tenant.
 - Organization: Your org.
 - Project: "ADF-Demo-Project".
 - Repository: "ADF-Repo".
 - Collaboration branch: "main".
 - Publish branch: "adf_{publish}".*Rootfolder : "/"*.
 - Import existing resources: Yes.
 - Save.
 - This initializes the repo with ADF artifacts.
5. Clone the repo locally:
 - In DevOps Repos, click "Clone" > Copy URL.
 - Use Git: `git clone <url>` in VS Code or terminal.
 6. Share the repo URL with the team. Inform Person 13 to proceed.

Time estimate: 15-20 minutes.

4.3 Person 13: Create the ADF Pipeline and Datasets in a Feature Branch

1. In ADF Studio, create a feature branch:
 - Top bar > Branch dropdown > "Create new" > Name: "feature/copy-pipeline" > From "main".
2. Create Linked Services:
 - Author tab > "+" > Linked service.
 - Blob Storage:
 - Name: "LS_{BlobStorage}".*Integrationruntime : AutoResolveIntegrationRuntime.*
 - Authentication: Account key.
 - Account: "adfdemostorage".
 - Key: Get from Portal > Storage > Access keys > key1.
 - SQL Database:
 - Name: "LS_{SQLDB}".*Integrationruntime : AutoResolve.*
 - Server: "adfdemosqlserver.database.windows.net".
 - Database: "adfdemodb".
 - Auth: SQL auth, User: "sqladmin", Pass: "P@ssw0rd123!".
3. Create Datasets:
 - "+" > Dataset > Azure Blob Storage > DelimitedText.

- Name: "DS_{InputCSV}".*Linkedservice* : LS_{BlobStorage}.
- File path: inputdata/sampleddata.csv.
- First row header: Yes.
- Schema: Import (Id int, Name string, Age int).
 - "+" > Dataset > Azure SQL Database > Table.
 - Name: "DS_{OutputSQL}".*Linkedservice* : LS_{QLDB}.
- Table: dbo.SampleData.
- Import schema.
- 4. Create Pipeline:
 - "+" > Pipeline > Name: "PL_{CopyData}".*Activities* > *Move&transform* > *Copydata*.
- Source: DS_{InputCSV}.*Sink* : DS_{OutputSQL}.
- Mapping: Auto-map.
- Validate and Debug (run once to test).
- 5. Publish changes:
 - Click "Publish all" > This commits to "feature/copy-pipeline".
- 6. In local Git (cloned repo):
 - `git checkout feature/copy-pipeline`
 - `git pull`
 - Verify JSON files: factory/adf-demo.json, linkedService/LS_{BlobStorage}.json, etc.
- 7. Inform Person 14 and team.

Time estimate: 20-25 minutes.

4.4 Person 14: Set Up the CI Build Pipeline in Azure DevOps

1. In Azure DevOps > Pipelines > New pipeline.
 - Where is code: Azure Repos Git > "ADF-Repo".
 - Configure: Existing Azure Pipelines YAML file? No, select "Starter pipeline".
 - Replace with YAML below, save as "azure-pipelines-ci.yml" in root.
2. YAML:

```
trigger:
  branches:
    include:
      - main
  paths:
    include:
      - '*'

pr:
  branches:
    include:
```

```

- main

pool:
  vmImage: 'ubuntu-latest'

steps:
- task: UseNode@1
  inputs:
    version: '16.x'

- script: npm install -g @microsoft/azure-data-factory-utilities
  displayName: 'Install ADF Utilities'

- script: |
  npx adf-validate "$(Build.Repository.LocalPath)" "adf-demo" "$(
    Build.Repository.LocalPath)/subscriptionId" "$(Build.
    Repository.LocalPath)/resourceGroup"
  displayName: 'Validate ADF'

- task: PublishBuildArtifacts@1
  inputs:
    PathToPublish: '$(Build.Repository.LocalPath)'
    ArtifactName: 'adfArtifacts'
    publishLocation: 'Container'
    displayName: 'Publish ADF Artifacts'

```

3. Note: This validates ADF JSON, requires npm package. For subscriptionId and resourceGroup, hardcode in YAML or use variables. For demo, replace "*\$(Build.Repository.LocalPath)/subscriptionId*" with "*Pipelines > Edit > Variables > Add*"subscriptionId", "resourceGroupName = adf - demo - rg".

4. Save and run once manually to test.

5. Inform Person 15.

Time estimate: 15-20 minutes.

4.5 Person 15: Set Up the CD Release Pipeline in Azure DevOps

1. In DevOps > Pipelines > Releases > New release pipeline.
 - Artifacts > Add > Source: Build pipeline (the CI one).
 - Stages > Add stage > Empty job.
 - Name stage: "Deploy to ADF".
2. Add tasks to stage:
 - Agent job > Agent pool: Azure Pipelines, ubuntu-latest.
 - Task 1: Azure PowerShell.
 - Display name: "Stop Triggers".
 - Azure connection: "AzureServiceConnection".
 - Script type: Inline.
 - Script:

```

$factoryName = "adf-demo"
$resourceGroup = "adf-demo-rg"
Get-AzDataFactoryV2Trigger -ResourceGroupName $resourceGroup
    -DataFactoryName $factoryName | Where-Object {$_.
    RuntimeState -eq 'Started'} | Stop-AzDataFactoryV2Trigger
    -ResourceGroupName $resourceGroup -DataFactoryName
    $factoryName -Force

```

- Task 2: Azure Resource Manager Template Deployment.
 - Display name: "Deploy ADF ARM".
 - Azure connection: "AzureServiceConnection".
 - Subscription: Your sub.
 - Action: Create or update resource group.
 - RG: "adf-demo-rg".
 - Location: East US.
 - Template: *(System.DefaultWorkingDirectory)/ADF-Repo/adfArtifacts/adf-demo/ARMTemplateForFactory.json(adjustpathbasedonartifact).Parameters:(System.DefaultWorkingDirectory)/adfArtifacts/adf-demo/ARMTemplateParametersForFactory.json*
- Override parameters: -factoryName "adf-demo" (add others if params exist).
 - Task 3: Azure PowerShell.
 - Display name: "Start Triggers".
 - Similar to stop, but use Start-AzDataFactoryV2Trigger.
- 3. Triggers: Continuous deployment > Enable on artifact from CI.
- 4. Save and create release manually to test.

Time estimate: 20-25 minutes.

4.6 Person 16: Test the Pipeline, Make a Change, Create Pull Request, Approve/Merge, and Validate

1. In ADF Studio or local Git, on feature branch, make a small change: e.g., add a description to the pipeline.
 - Commit and push: `git add . git commit -m "Add description" git push origin feature/copy-pipeline`
2. In DevOps Repos > Pull requests > New PR: From feature/copy-pipeline to main.
 - Title: "Add copy pipeline".
 - This triggers CI build.
3. Approve and complete PR (merge).
4. Merge triggers CD release. Monitor Releases.
5. After deploy, in ADF Studio (on main), run the pipeline manually.
 - Verify data in SQL: Query `SELECT * FROM SampleData;` should show rows.

6. Clean up if needed: Delete resources after demo.

Time estimate: 15-20 minutes.

5 Collaboration Notes

- Coordinate via chat or calls for dependencies (e.g., Person 11 shares credentials before Person 13 starts).
- If issues, check permissions, service connection.
- This completes a full CI/CD for data pipeline, showing lifecycle from dev to prod.